

UNIVERSITY OF PLYMOUTH

**COMP3004 Advanced Computing and Networking
Infrastructures**

Assessment 2: Software Defined Networking

Reece Davies [10572794]

BSc (Hons) Computer Science

Abstract

This document investigates how different network conditions affect end-to-end video streaming quality for an Adaptive Video Streaming web-based service. In order to make such judgements, a Testbed platform is created, which specifically relies upon the services: DASH.js, Apache server, Mininet, and OpenDaylight. Furthermore, in order to demonstrate the development process of creating the video streaming system, detailed explanations and illustrated figures are provided. Subsequently, statistical results are presented for the various experiments carried out in testing the system under different network conditions. These results are analysed and discussed to reach a respectable conclusion of what conditions prove to be most effective for an Over-The-Top media streaming service.

1. Introduction

In this day and age, internet video is the dominant source of internet traffic generated. This therefore means that when applying video streaming to a software defined network (SDN), it is crucial that both the quality and efficiency of the content is at its peak to ensure that viewers remain satisfied. Of course, network conditions can be variable, especially when conducted through a wireless connection. As a result, making slight amendments to the characteristics of the content will ultimately affect the performance of the video player in question.

2. Testbed Setup, configuration and code developed

2.1 Install VM

The entirety of this demonstration is conducted on an “Ubuntu Linux Server 18.04.5” Virtual Machine (VM) with the use of the software “VM Workstation”. The ISO file used for Ubuntu Server 18 is available at:

<https://releases.ubuntu.com/18.04/>

2.2 Mininet

Mininet is installed on the Ubuntu VM to simulate the tasks for SDN video streaming; this software is a network emulator that creates a network of virtual hosts, switches, controllers, and links. The Linux package for Mininet can be installed with the command `sudo apt-get install mininet`. Additionally, an already configured Mininet VM file is provided through Mininet’s official website, which uses the “Mininet VM installation” file, available at:

<https://www.vmware.com/products/workstation-player.html>

Regardless of method, it is important that the VM’s network adapter is set to “Bridged” (see figure 1), as this will allow for interconnectivity between the different services because the VM is automatically connected to the host computer’s network.

Device status

☐ Connected

☒ Connect at power on

Network connection

☒ Bridged: Connected directly to the physical network

☐ Replicate physical network connection state

☐ NAT: Used to share the host's IP address

☐ Host-only: A private network shared with the host

☐ Custom: Specific virtual network

VMnet0

☐ LAN segment:

LAN Segments... Advanced...

Figure 1: VM Workstation network settings

2.3 OpenDaylight Controller

Once the Mininet VM has been setup and configured, a new VM must be created, which will be used as the network's Controller. OpenDaylight (ODL) is a piece of software that enhances SDN by customising and automating networks. As ODL is implemented solely as software and kept within its separate Java Virtual Machine, it can be implemented on any operating system that supports Java. To download ODL directly to the VM, use the `wget` command, which retrieves the associated `tar.gz` file for ODL. The full command is as follows:

```
sudo wget
https://nexus.opendaylight.org/content/repositories/public/org/.opendaylight/integration/karaf/
0.8.2/karaf-0.8.2.zip
```

Once this file has been downloaded, it can be unzipped with the command `unzip karaf-0.8.2.zip`. To launch the ODL application, the executable file `karaf` must be run, which is located within the downloaded `karaf-0.8.2` file in the `bin` directory, displayed in figure 2.

```

TX packets:48 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:32951 (32.9 KB) TX bytes:4622 (4.6 KB)

lo
Link encap:Local Loopback
inet addr:127.0.0.1 Mask:255.0.0.0
inet6 addr: ::1/128 Scope:Host
UP LOOPBACK RUNNING MTU:65536 Metric:1
RX packets:16 errors:0 dropped:0 overruns:0 frame:0
TX packets:16 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:0
RX bytes:1184 (1.1 KB) TX bytes:1184 (1.1 KB)

odl@odl:~$ ls
Downloads odl
odl@odl:~$ cd odl/bin/
odl@odl:~/odl/bin$ ls
client          custom_shard_config.txt  karaf          setenv.bat     start          status.bat
client.bat      instance                karaf.bat      shell          start.bat     stop
configure_cluster.sh instance.bat             setenv         shell.bat      status        stop.bat
odl@odl:~/odl/bin$ ./karaf
Java HotSpot(TM) 64-Bit Server VM warning: ignoring option MaxPermSize=512m; support was removed in 8.0

Hit '<tab>' for a list of available commands
and '[cmd] --help' for help on a specific command.
Hit '<ctrl-d>' or type 'system:shutdown' or 'logout' to shutdown OpenDaylight.

opendaylight-user@root>

```

Figure 2: Launching OpenDaylight

2.4 Apache Server on One Virtual Host

Subsequently, Apache Server must be installed onto the virtual host (the Mininet VM), and this will allow the Mininet VM to behave as a web server, thus allowing other devices to access it for the relevant video streaming capabilities.

This can be accomplished with the command `sudo apt-get install apache2`. Following installation, the user can verify if it is running with the command `ps -aux | grep apache2`, or simply by visiting the VM's IP address in a web browser (see figure 3). If apache is not running, the user can manually start it with the command `apache2ctl start`, or alternatively `systemctl start apache2`.

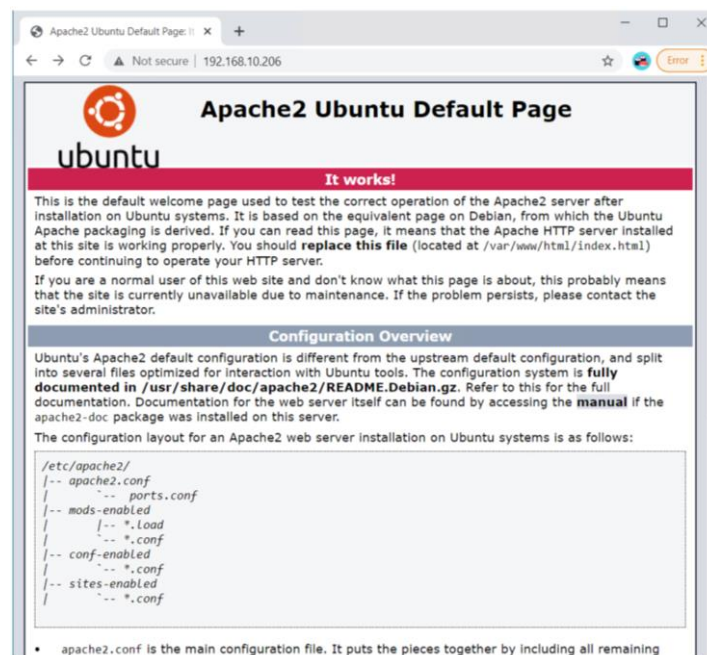


Figure 3: Apache server index page

2.5 DASH.js Player

Dash.js is an opensource Media Extensions player that also functions as the reference client for DASH IF (Industry Forum). In this demonstration, Dash player was downloaded from a private server, owned by Plymouth University; the command `sudo wget 10.224.41.8/bbb/dashjs.zip` downloaded a zip file containing the Dash.js reference client. It was important that this command was executed in the `var/www/html` directory because this is Apache's default root directory for the server's displayed contents.

Once the zip file was downloaded, it must then be unzipped with the command `sudo unzip dashjs.zip`. After the file has been unzipped, Dash player can be accessed by inputting the server's IP address in a web browser, followed by `/dashjs`, as displayed in figure 4.

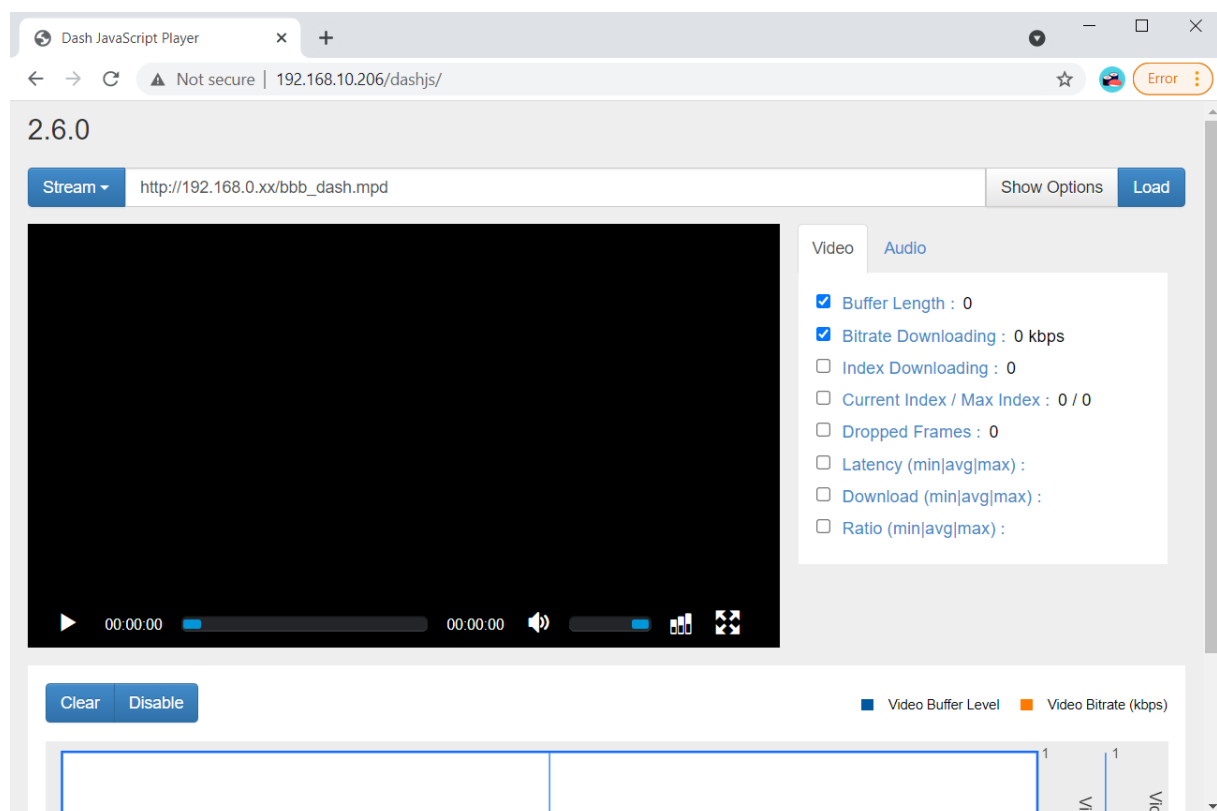


Figure 4: Dash player's start-up page

3. Build a Network Topology with Mininet

Firstly, the Mininet VM must be remotely logged in with SSH (Secure Shell) from MobaXterm – a windows application that behaves as an advanced tool for remote computing. By making use of MobaXterm in this demonstration, it therefore allows for the use of windows tabs for the Mininet VM, which does not host a GUI (Graphical User Interface). Figure 5 displays MobaXterm being used to remotely log into the Mininet VM. In order for the VM to make use of MobaXterm's features, the Linux package `xterm` must be installed with the `sudo apt-get install xterm` command.

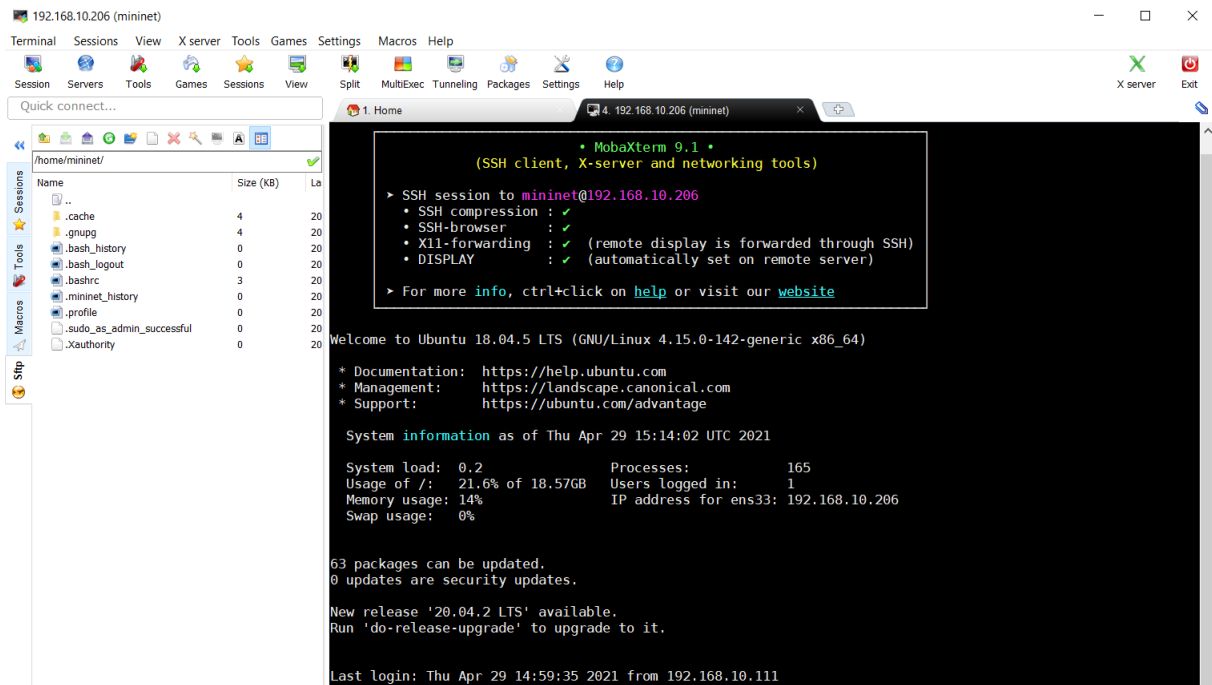


Figure 5: Using SSH to log into Mininet VM via MobaXterm

Subsequently, Mininet is used to create the network topology, which can be done by executing the following command from MobaXterm:

```
sudo mn -x --mac --topo single,3 --controller=remote,ip=192.168.10.239,port=6633 --switch ovsk,protocols=OpenFlow13
```

To implement the ODL controller, the **--controller** option is added, alongside the ODL VM's IP address (**192.168.10.239**). This therefore allows for interconnectivity between the different nodes in the network, and automation of the devices' functions.

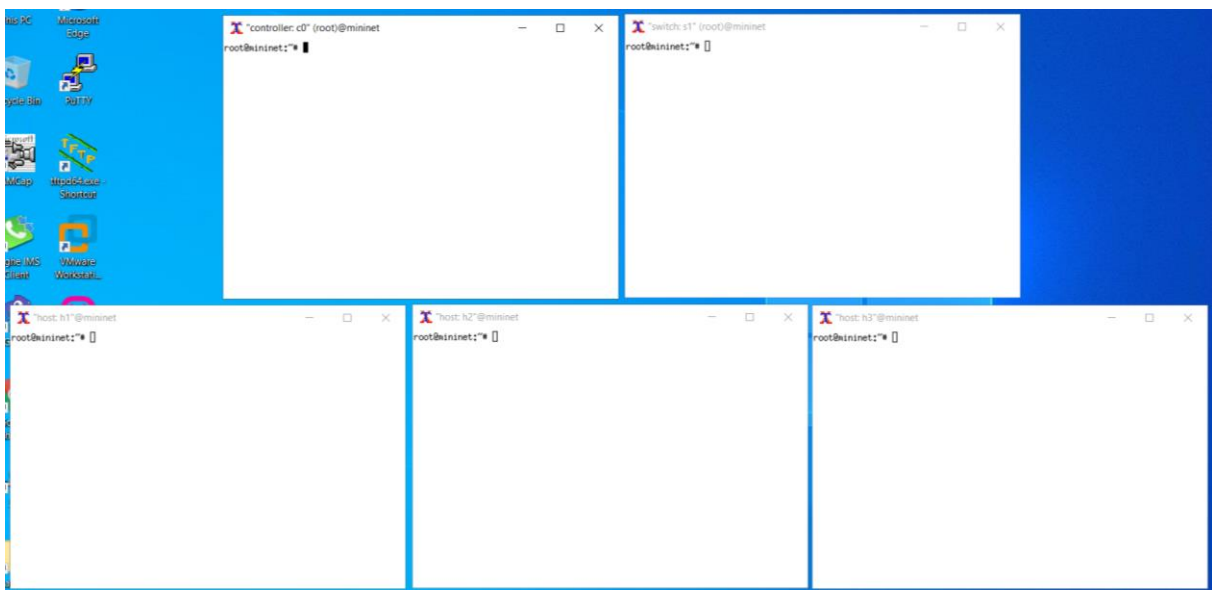


Figure 6: CLI of each node in topology

If the `mn` command is a success (alongside the implementation of the `-x` option that makes use of xterm), a window should be opened for each device in the topology, as shown in figure 6. As the `--mac` option was used, h1 has an IP address of **10.0.0.1**, whilst h2 has **10.0.0.2**, and h3 of **10.0.0.3**, respectively. Alternatively, Mininet topology can be created with the use of a python script, displayed in figure 7.

```
def emulate(depth_, fanout_):  
    # Create a network with tree topology  
    tree_ = TreeTopo(depth=depth_, fanout=fanout_)  
  
    # Initiating the Mininet instance  
    net = Mininet(topo=tree_)  
  
    # Start Execution of the Emulated System.  
    net.start()  
  
    # Name two of the instances as h1 and h2.  
    h1, h2 = net.hosts[0], net.hosts[depth_]  
  
    # Ping from an instance to another, and print the output.  
    print (h1.cmd('ping -c1 %s' % h2.IP()))  
  
    # Stop the Mininet Emulation.  
    net.stop()
```

Figure 7: Python script for mn

4. Prepare the Video

4.1 Install Required Packages

This demonstration uses a video that is supplied by Plymouth University to show Dash's streaming capabilities. Prior to downloading the video, two tools must be installed onto the Mininet VM, and are responsible for conforming the video to Dash.js's requirements; these tools are X264 (responsible for encoding) and MP4Box (for segmentation). The relevant packages for X264 can be installed with `sudo apt-get install x264`, and the packages of MP4Box with the `sudo apt-get install gpac`.

4.2 Download the Video

Once the packages have been installed, the video must be downloaded to Apache's root directory `var/www/html`, which is accomplished with the command `sudo wget 10.224.41.8/bbb/bbb1.mp4`. The video **bbb1.mp4** is nearly 8 minutes long and has a resolution of 1280x720; as a result, it has a file size of 146 Megabytes. The process of encoding and segmenting the video can potentially be time consuming, especially if the chosen video has a large file size. In an effort to reduce the time required for these processes, the length of the video is shortened to two minutes, resulting in the reduction of the video's file size. In order to achieve this, **ffmpeg** must be installed, which is done with the command `sudo apt-get install ffmpeg`.

Subsequently, the command `sudo ffmpeg -i bbb1.mp4 -t 00:02:00 bbb1_2m.mp4` creates a duplicate of **bbb.mp4**, however limited to the first two minutes. This new file is saved as **bbb1_2m.mp4**, but to eliminate any confusion, the original file is then replaced with this new shorter video by executing the command `sudo mv bbb1_2m.mp4 bbb1.mp4`.

4.3 Encode & Segment the Video

The video must be encoded to **H.264/AVC** which therefore allows for the video to be playable by the Dash.js player. This is achieved by X264 with the command:

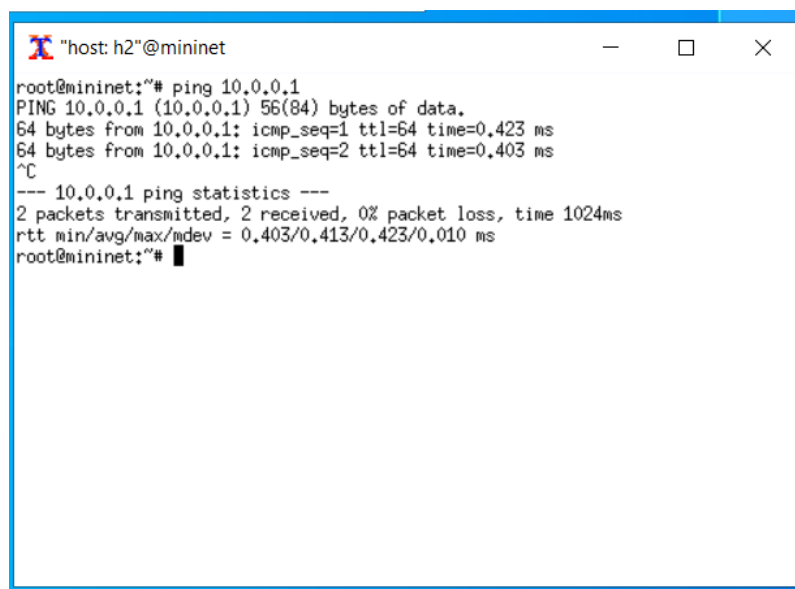
```
sudo x264 --output bbb_2400k.264 --fps 30 --bitrate 2400 --video-filter  
resize:width=1280,height=720 bbb1.mp4
```

Once the video has been encoded, the created **H.264** raw video must be added to an mp4 container, as this is the container format of choice for this demonstration. This is done by MP4Box with the command `sudo MP4Box -add bbb_2400k.264 -fps 30 bbb_2400k.mp4`, where **bbb_2400k.mp4** is the name of the file being created.

The video is then segmented by executing the command `sudo MP4Box -dash 4000 -frag 4000 -rap -segment-name segment_2400k_ bbb_2400k.mp4`. As a result, the video is separated into manageable partitions for the video player, which therefore allows for efficient load times.

4.4 Stream Video within Network Topology

To ensure that the separate nodes within the topology are able to communicate to one another, the ping command is used; e.g. executing `ping 10.0.0.1` from h2 verifies that it can communicate with the h1 node (displayed in figure 8).



```
"host: h2"@mininet  
root@mininet:~# ping 10.0.0.1  
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.  
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.423 ms  
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=0.403 ms  
^C  
--- 10.0.0.1 ping statistics ---  
2 packets transmitted, 2 received, 0% packet loss, time 1024ms  
rtt min/avg/max/mdev = 0.403/0.413/0.423/0.010 ms  
root@mininet:~#
```

Figure 8: Pinging h1 from h2

As h1 is used as the host, it is important that Apache server is running on this node, which is achieved by executing the command `apache2ctl -k start` from h1. To stream the video from within the topology, Firefox is loaded by running the command `sudo firefox`. If Firefox is not installed, this can be accomplished with the command `sudo apt install firefox`.

Once Firefox is running, h1's Dash.js can be accessed from h2, where the video will be streamed (displayed in figure 9). The file selected must be the dash file created by MP4Box; in this case, it is `bbb_2400k_dash.mpd`.

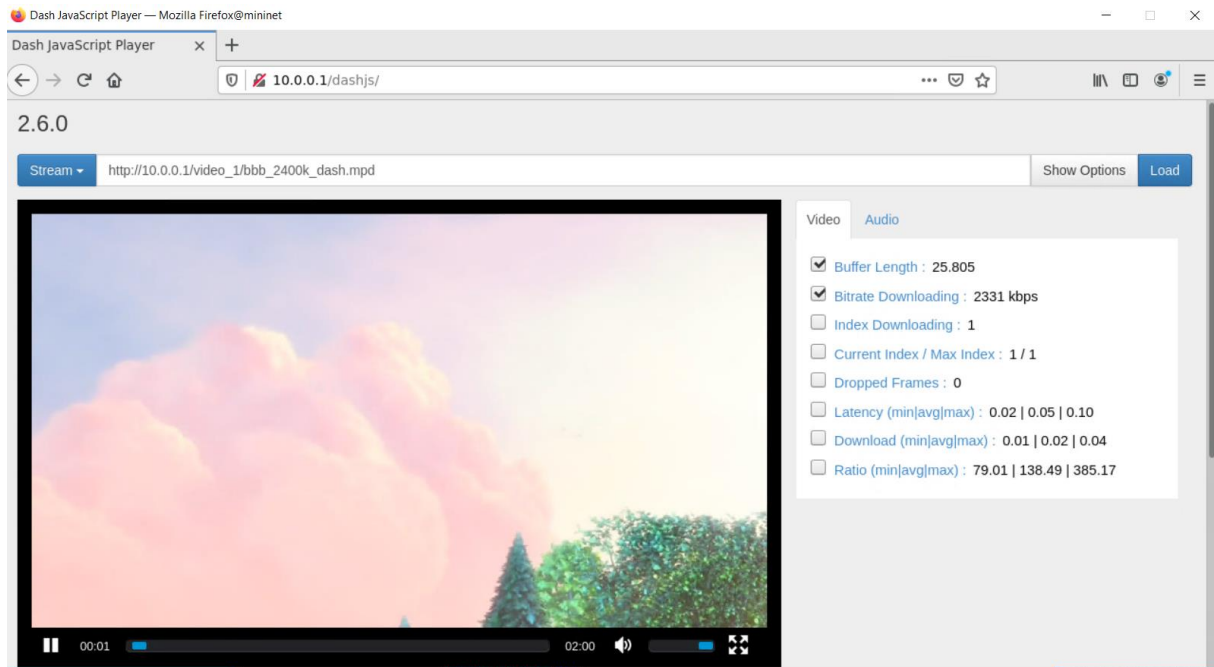


Figure 9: Playing Dash from h2

4.5 Automate the Processes

The processes from section 4.4 are repeated with two more video representations, each containing slightly different specifications, however a bash script is implemented to automate the encoding and segmenting processes. A new directory is created, where the second video's files will be stored (**video_2**); the bash script is created with the command `nano automate_video_2` and then made executable with `sudo chmod +x automate_video_2`. The same is done with the third video, and the scripts' contents are displayed in figures 10 and 11, respectively.

```
#!/bin/bash
sudo x264 --output bbb_1200k.264 --fps 30 --bitrate 1200 --video-filter resize:width=1920,height=1080 ../bbb1.mp4
sudo MP4Box -add bbb_1200k.264 -fps 30 bbb_1200k.mp4
sudo MP4Box -dash 4000 -frag 4000 -rap -segment-name segment_1200k_ bbb_1200k.mp4
```

Figure 10: Second video's bash script

```
#!/bin/bash
sudo x264 --output bbb_600k.264 --fps 30 --bitrate 600 --video-filter resize:width=2560,height=1440 ../bbb1.mp4
sudo MP4Box -add bbb_600k.264 -fps 30 bbb_600k.mp4
sudo MP4Box -dash 4000 -frag 4000 -rap -segment-name segment_600k_ bbb_600k.mp4
```

Figure 11: Third video's bash script

The traits of each video have been slightly adjusted to demonstrate how altering the network conditions might affect these videos differently. The specifications are displayed in the following table.

	Video 1	Video 2	Video 3
Bitrate	2400	1200	600
Resolution	1280x720	1920x1080	2560x1440
FPS	30	30	30

Video one is given normal traits, however the following two videos are of higher quality, in terms of pixel resolution, whilst also simultaneously decreasing their maximum bitrate value.

5. Experiments

5.1 Default Network Conditions

The first test is conducted with Mininet's default network conditions, which does not restrict the bandwidth within the topology. To demonstrate this, the bandwidth is measured with the `iperf` command. h1 executes the command `iperf -s` followed by h2 executing the command `iperf -c 10.0.0.1`. The results are displayed in figure 12.

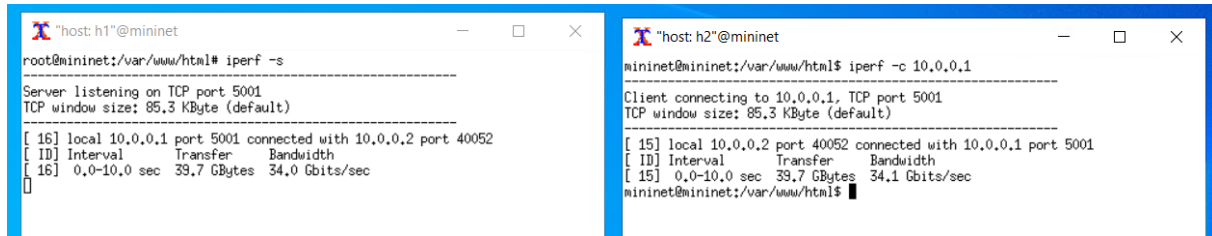


Figure 12: Measuring bandwidth in default network conditions

The three different videos were streamed under these conditions, with a snapshot taken of the statistics of each video after sixty seconds of playtime.

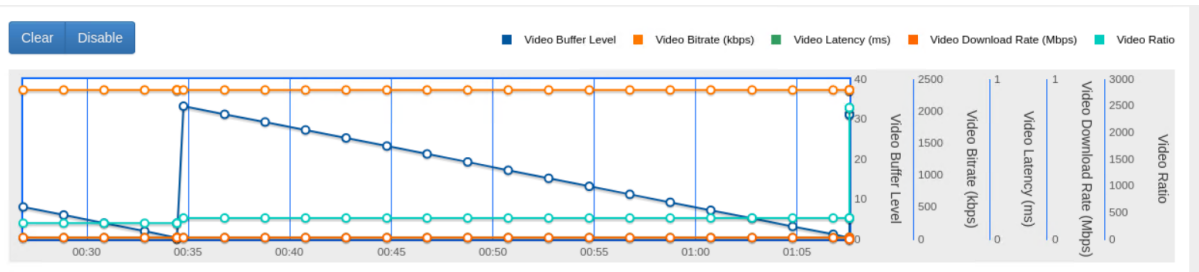


Figure 13: Video 1 (default network)

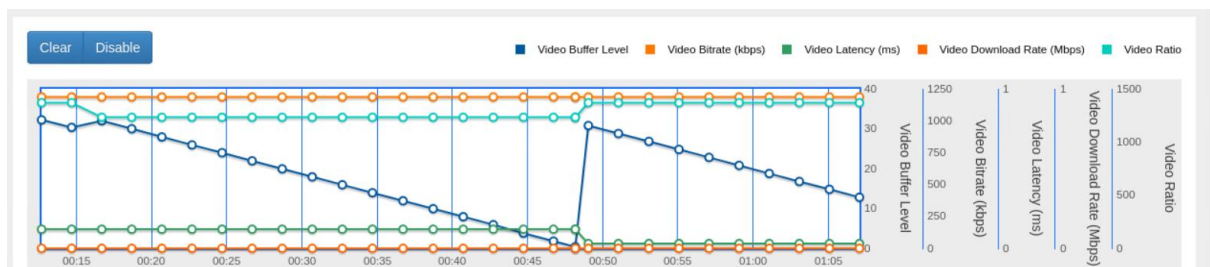


Figure 14: Video 2 (default network)

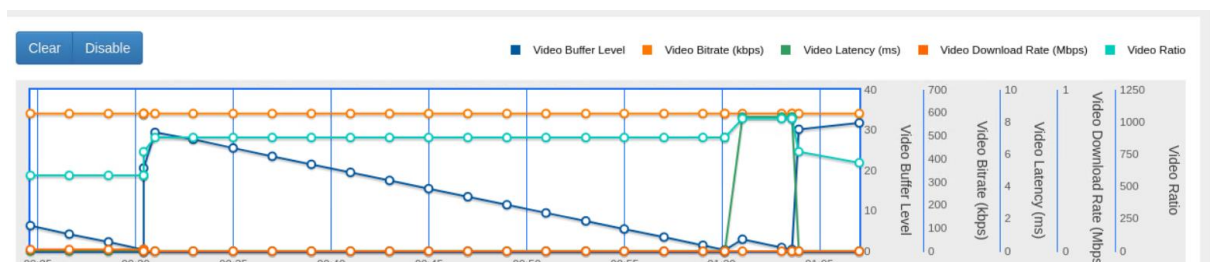


Figure 15: Video 3 (default network)

5.2 Medium Network Conditions

The test is conducted again, although altering the bandwidth and delay of the topology to what would be considered somewhat slow broadband. In this test, the bandwidth had been set to 50 with a delay of 10ms. `--link tc,bw=50,delay=10ms` is added to the `mn` command provided in section 3.

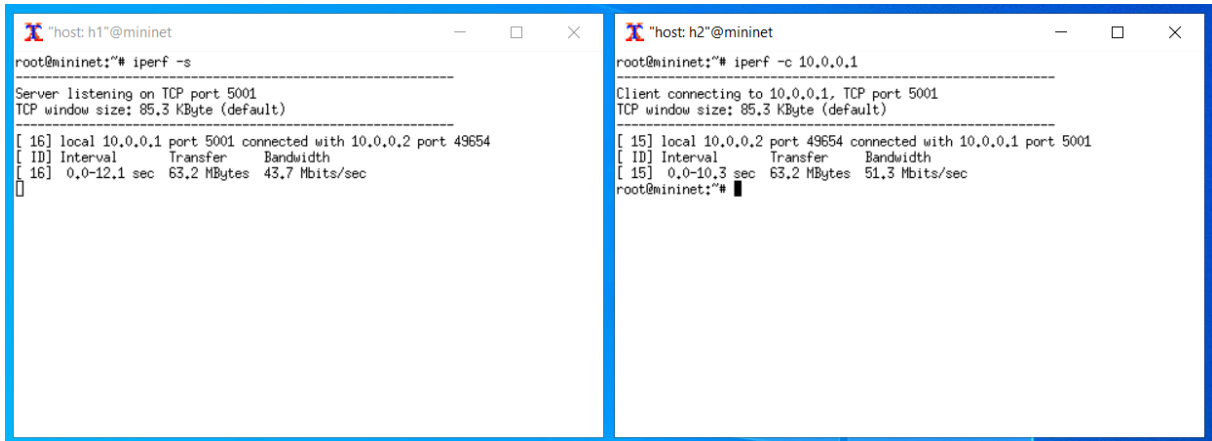


Figure 16: Measuring bandwidth in medium network conditions

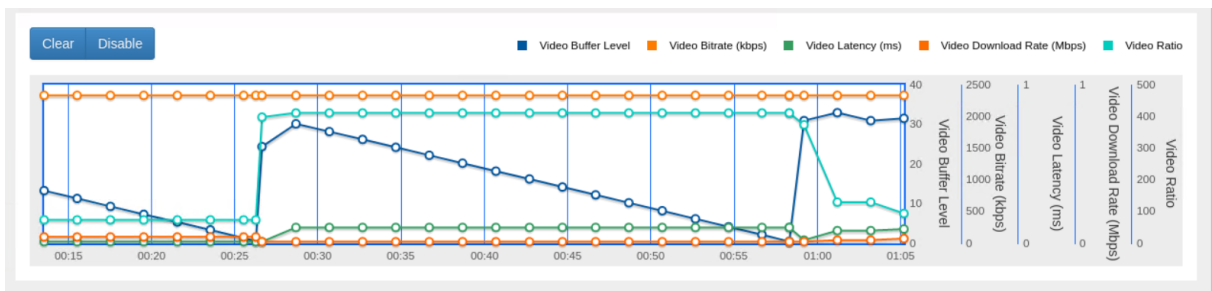


Figure 17: Video 1 (medium network conditions)

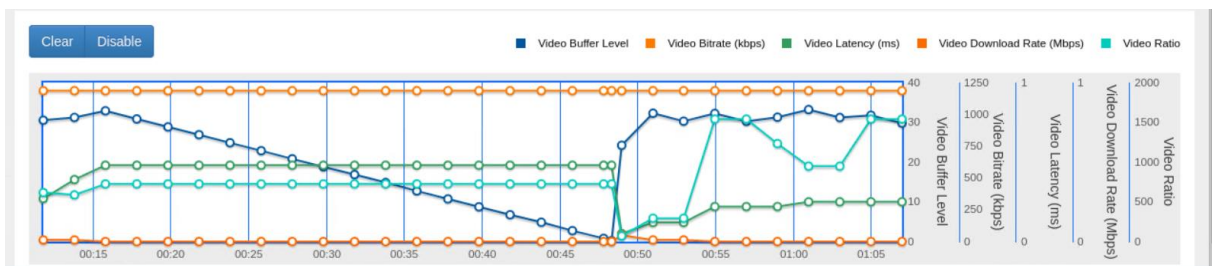


Figure 18: Video 2 (medium network conditions)

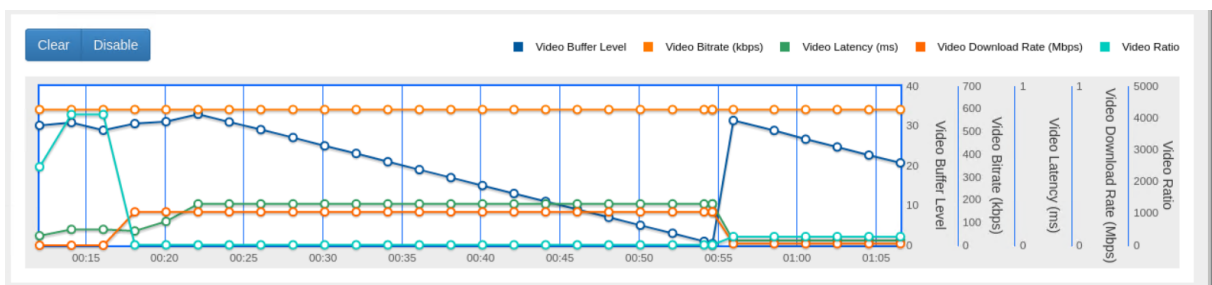


Figure 19: Video 3 (medium network conditions)

5.3 Slow Network Conditions

The experiment is conducted one final time, except with network conditions that are considered extremely slow. Originally, the plan was set the bandwidth to 1 with a delay of 50ms, however proved to be inadequate for video streaming, as Dash player would refuse to successfully load. Instead, the networking conditions were slightly altered to be a bandwidth of 10 with a delay of 30ms.

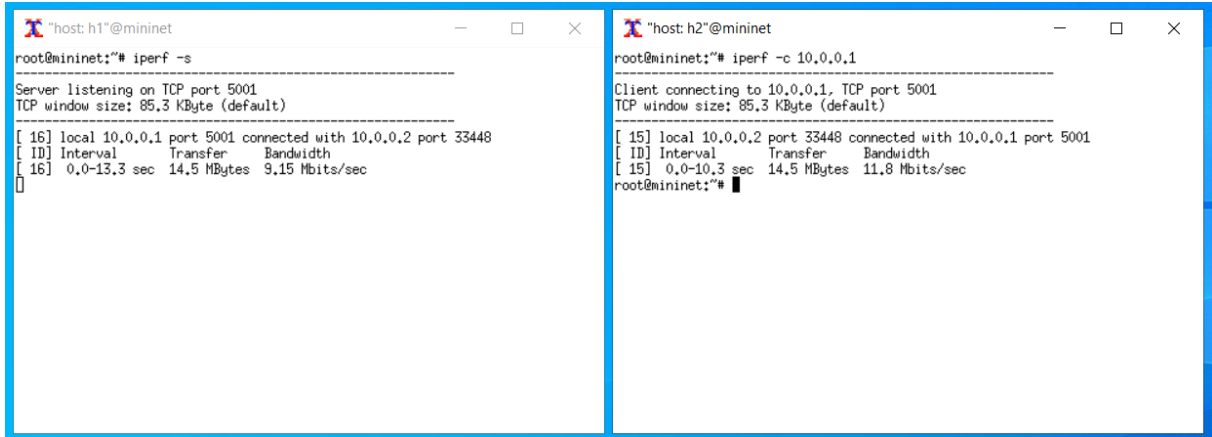


Figure 20: Measuring bandwidth in slow network conditions

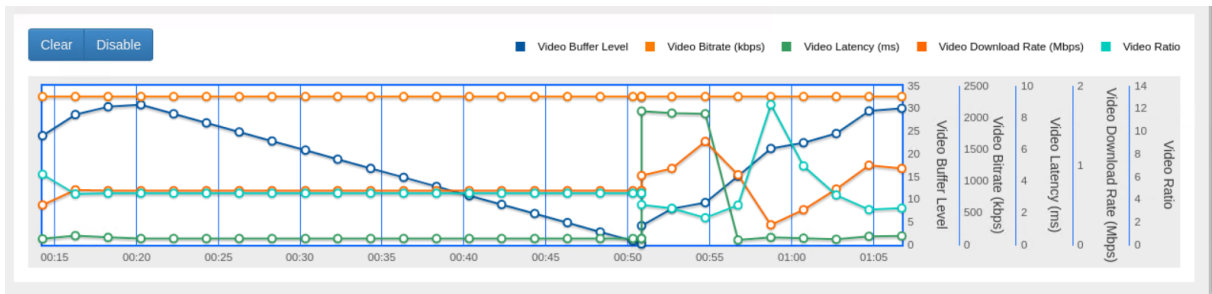


Figure 21: Video 1 (slow network conditions)

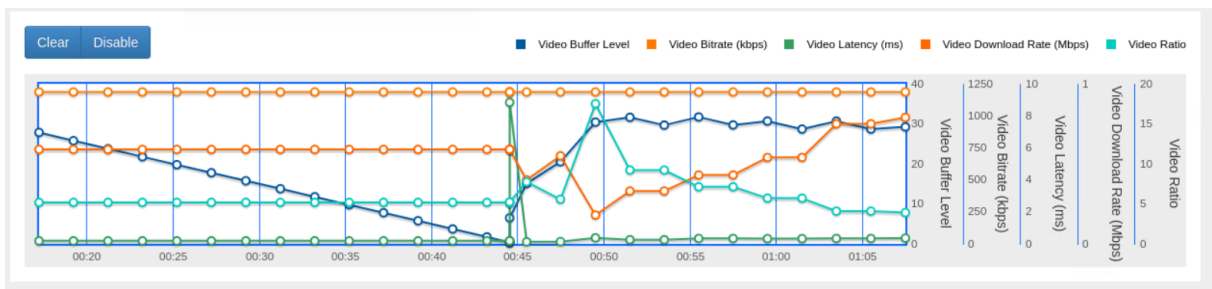


Figure 22: Video 2 (slow network conditions)

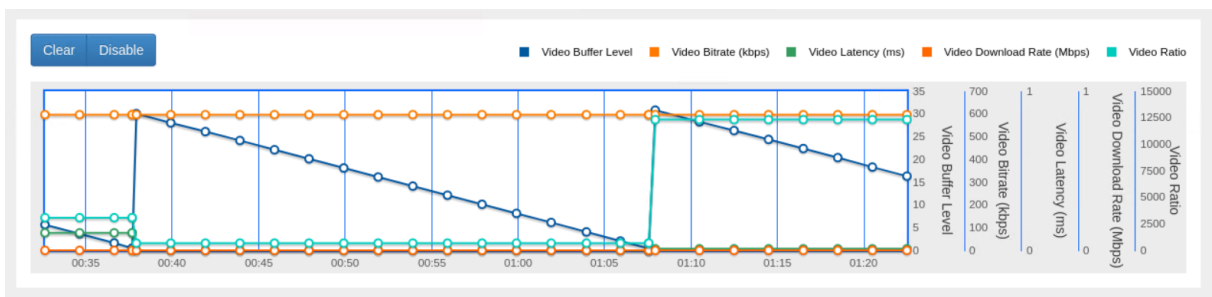


Figure 23: Video 3 (slow network conditions)

5.4 Discussion of Results

The figures displayed from each of the three experiments clearly show how playback is affected by different network conditions. Each test recorded the video's buffer level, bitrate, latency, download rate, and ratio. In all the tests, the video's bitrate remained stable, all of which were between 30 and 40 kbps. With default network conditions, the download rate remained low, however became unstable with a slow bandwidth. The same can be applied to the bitrate, whereas the bandwidth decreased, so did the stability of the video's bitrate. It was common for it to be the same value within the first forty seconds of the video, where afterwards it would suddenly fluctuate.

With regards to latency, it appeared to remain low throughout the first and third videos for all tests, however this cannot be said for video two. The latency stayed somewhat high until reaching the one minute mark; this can potentially be due to the sudden rise of the video's buffer levels, which meant less bandwidth was required for loading the video for the client. Furthermore, at the start of each video, the buffer levels were initially high, but followed a slow decline as the videos were being played. These results clearly show that each video was being loaded faster than it was being played; however, amongst all tests, the buffer level had a sudden spike prior to reaching one minute of streaming. This is most likely due to the following video segments being processed for the Dash player.

Lastly, the video's ratio seemed to follow similar behaviours to the other recordings, where it would remain stable until reaching one third of the video's length; it is at this time when the buffer level has a sudden spike. Other than the statistical recordings, one aspect that became particularly clear is how each video was played back. Upon altering the network conditions to have decreased bandwidth, the videos became somewhat 'choppy', therefore potentially leading to an unpleasant experience for any viewers (shown in figure 24).

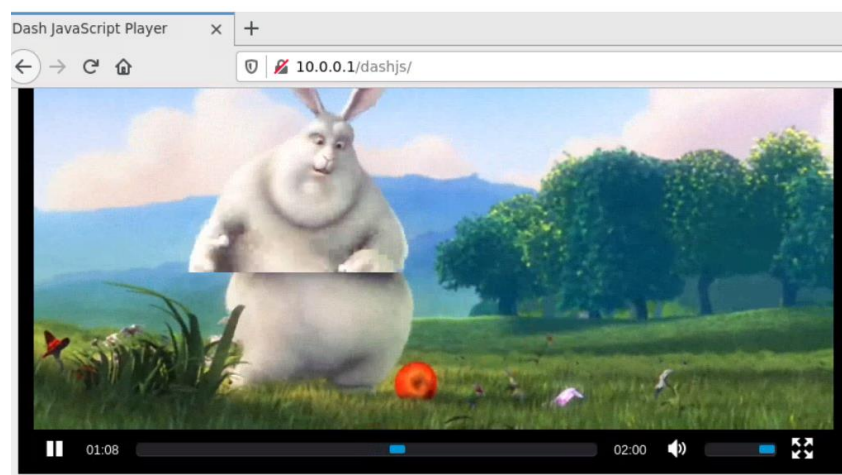


Figure 24: 'Choppy' playback

6. Improve Video Streaming in SDN

This demonstration has shown how the statistics of video streaming can be somewhat instable, especially with regards to the bitrate. What's worse, is that there can be unfairness between different clients, where the total bandwidth available is not utilised by the total bandwidth generated by all the clients. SDN is a network architecture that aims to separate 'control' aspect from the data. By implementing a controller into a network, it allows for real-time adjustments to be made to the network's data.

In a common home network, multiple clients are connected to the network through the gateway, however when numerous video flows compete for the available bandwidth, this could result in poor

performance. One solution is presented by Abuteir, R. Fladenmuller, A. Fourmaux, O (2016), where a new framework is generated – Network Assisted Video Streaming (NAVS), which is based on standard SDN architecture. This new architecture utilises a controller that consists of five different modules: Policy, Traffic Monitoring, MPD Analyser, QoE Optimiser, and Traffic Shaping.

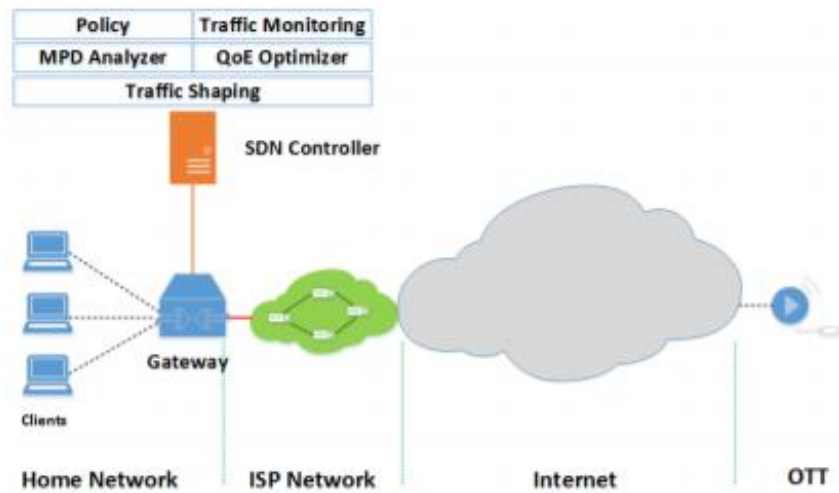


Figure 25: NAVS architecture for home network (Abuteir, R. Fladenmuller, A. Fourmaux, O. (2016))

Specifically focusing on the traffic monitoring, MPD analyser, and QoE optimiser modules, the implementation of these concepts can significantly enhance the performance of video streaming. The traffic monitoring module is responsible for monitoring all traffic within the given network, and when a new client starts using the network, the first package sent by the client is forwarded from the gateway to the controller. This therefore allows the controller to monitor new traffic flows, however when a flow is classified as video flow, it is added to a list of active video flows, which done by the MPD analyser module.

Based on these predefined policies, the QoE optimiser module calculates the optimal allocation of bandwidth between numerous clients engaging in video streaming. If it is known that client n wants to stream video at bitrate Rn , it therefore requires the bandwidth $(1 + \alpha Rn)Rn$, therefore it is possible to formulate an optimisation problem for N users:

$$\arg \max \sum_{n=1}^N B_n$$

Subject to:

$$\sum_{n=1}^N B_n \leq BW$$

$$B_i = B_j \quad \forall \text{ two video flows } i \text{ and } j$$

Where video flow n needs bandwidth $Bn = (1 + \alpha Rn)Rn$ to stream at bitrate Rn (selected by user n) and αRn is the extra bandwidth needed by the flow that stream video with bitrate Rn . This extra bandwidth is added due to the required protocol overhead, queuing, and retransmission. This

'objective' function of the QoE optimiser is to maximise the bitrate that each client receives when the traffic is related to video traffic. As a result, it ensures that the performance of each client's video player is defined by to their desired bitrate R_n , and therefore reduces the possibility of instability of the client's player. This also assists the fairness of video flows, where any instability generated may somewhat promote unfairness between the players' requested bitrate and the network's bandwidth under-utilisation.

Conclusions

Overall, this experiment has clearly demonstrated how altering a network's conditions directly affects the performance of video streaming over a SDN. Although, the statistical values retrieved from such experiments are not fully examined, visual representations are presented in figures to demonstrate the behaviours that is a direct result of reducing a network's bandwidth. It is also apparent that creating the new architecture NAVS may significantly improve the performance of video streaming in home networks. NAVS allocates the available bandwidth for different video streams based on the network's policy, number of video flows, and available bitrate, therefore resulting in the evaluation of the most appropriate bandwidth for each competing client under the same network.

References

A. Barakabitze, A. Barakabitze, A. Bentaleb, A. Bentaleb, A. Fourmaux, A. Zinner, A. Zinner, B. (2019). BBGDASH: A Max-Min Bounded Bitrate Guidance for SDN Enabled Adaptive Video Streaming. IEEE: New York., B. (2019). Bandwidth Prediction Schemes for Defining Bitrate Levels in SDN-enabled Adaptive Streaming. IEEE: New York

Al-Issa, E. (2014). What Is an OpenDaylight Controller? AKA: OpenDaylight Platform. SDX Central. Viewed 6th Apr 2021. <<https://www.sdxcentral.com/networking/sdn/definitions/opendaylight-controller/>>

Sobanski, J. (2017). SDN-based throughput allocation in wireless networks for heterogeneous adaptive video streaming applications. IEEE: New York.

Cisco. (2021). What Is a Network Controller?. Cisco Systems. Viewed 18th May 2021. <<https://www.cisco.com/c/en/us/solutions/enterprise-networks/what-is-a-network-controller.html>>

Al-Issa, J. (2018). How to install OpenDaylight on CentOS 7 or Ubuntu 16.04 LTS. John Sobanski. Viewed 15th May 2021. <<https://john.sobanski.com/how-to-install-opendaylight-on-centos-or-ubuntu.html>>

Dsilhavy. (2021). Dash-Industry-Forum. GitHub. Viewed 15th May 2021. <<https://github.com/Dash-Industry-Forum/dash.js/wiki>>

MobaXterm. (2021). MobaXterm. MobaXterm Mobatek. Viewed 15th May 2021. <<https://mobaxterm.mobatek.net/>>

Mininet. (2021). Mininet Walkthrough. Mininet. Viewed 16th May 2021. <<http://mininet.org/walkthrough/>>

Mininet. (2021). Automating Controller Startup. Mininet. Viewed 16th May 2021. <<http://mininet.org/blog/2013/06/03/automating-controller-startup/>>

Abuteir, J. Lloret, L. Jimenez, M. Garcia, Mininet. (2021). Mininet Overview. Mininet. Viewed 1st May 2021. <<http://mininet.org/overview/>>

Moriarty-Siler, O. (2016). SDN Based Architecture to Improve Video Streaming in Home Networks. IEEE: New York.

Taha, R. Fladenmuller, T. Ghita, T. Ghita

Appendix

```
def emulate(depth_, fanout_):  
  
    # Create a network with tree topology  
    tree_ = TreeTopo(depth=depth_, fanout=fanout_)  
  
    # Initiating the Mininet instance  
    net = Mininet(topo=tree_)  
  
    # Start Execution of the Emulated System.  
    net.start()  
  
    # Name two of the instances as h1 and h2.  
    h1, h2 = net.hosts[0], net.hosts[depth_]  
  
    # Ping from an instance to another, and print the output.  
    print (h1.cmd('ping -c1 %s' % h2.IP()))  
  
    # Stop the Mininet Emulation.  
    net.stop()
```